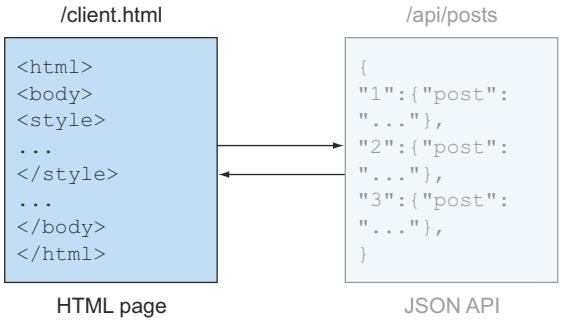


Figure 3.5 Location of the client web page within the sample app

JavaScript, and will be used to load the posts from the API. Next, the code defines an `onload` function that executes when the HTTP response is received. This particular function parses the blog posts into a JSON object, then displays the posts on the page. After the request behavior is configured, the actual request is sent to the server. The `getBlogPost` function is called when the page loads, so that the posts are automatically displayed when the user visits the page. If there is an error when making the HTTP request, the page displays the word `'ERROR'` on the page to let you know that something is wrong.

Listing 3.2 Sample client code

```
<!DOCTYPE html>
<html><body onload="getBlogPosts();">
<style>
.post {margin-bottom: 20px;}
</style>
<div id="output"></div>
<script>
var createXhr = function(method, url) {
  var xhr = new XMLHttpRequest();
  xhr.onerror = function() {
    document.getElementById('output').innerHTML = 'ERROR';
  };
  xhr.open(method, url, true);
  return xhr;
};

var getBlogPosts = function() {
  var xhr = createXhr('GET', 'http://127.0.0.1:9999/api/posts');
  xhr.onload = function() {
    var data = JSON.parse(xhr.responseText);
    var elem = document.getElementById('output');
    for (var postId in data) {
      var postText = data[postId]['post'];
      var div = document.createElement('div');
      div.className = 'post';
      div.appendChild(document.createTextNode(postText));
      elem.appendChild(div);
    }
  };
```

Gets posts when page loads

Displays an error if request fails

Creates a new XMLHttpRequest object

Displays posts on page